

אוניברסיטת תל אביב

הפקולטה להנדסה ע"ש איבי ואלדר פליישמן

דוח מסכם לפרויקט גמר

Gesture-Controlled Car

מעבדה חשיפה לחומרה (0512-4013)

שם הקורס:

רז טסה – 322382755

מגישים:

ווסים בכריה – 213651334

משה פלג

מנחה:

18 במרץ 2026

תאריך:

תוכן העניינים

3	1 תקציר
3	2 מבוא
3	3 רקע תאורטי
4	4 מהלך הניסוי
5	5 תוצאות וניתוח
6	6 קשיים
6	7 סיכום ומסקנות
6	8 כיווני המשך
7	9 נספחים – קודי המערכת
7	9.1 קוד המשדר (TSM.ino)
10	9.2 קוד המקלט (RCV.ino)

1. תקציר

פרויקט זה מציג תכנון ומימוש של מערכת בקרה אלחוטית עבור רכב רובוטי, המבוססת על מחוות יד (Gesture Control). המערכת מאפשרת ניהוג מרחוק באמצעות הטיית כף היד בלבד, ללא צורך בממשקים פיזיים כגון ג'ויסטיק או כפתורים.

המערכת מבוססת על שתי יחידות קצה: משדר הליביש על כף היד ומקלט המותקן על שלדת רכב. המשדר עושה שימוש בחיישן אינרציאלי מדגם MPU6050 לניתוח זוויות ההטיה, והמידע מועבר אלחוטית באמצעות מודול NRF24L01. המקלט מפעיל מערך של שני מנועי DC דרך בקר כוח L298N. התוצאה הסופית היא מערכת יציבה המגיבה בזמן אמת למחווות המשתמש.

2. מבוא

בעולם הטכנולוגי המודרני, הממשק בין אדם למכונה (HMI) הופך לאינטואיטיבי יותר. פרויקט זה נולד מתוך השאיפה להחליף את אמצעי השליטה המסורתיים במערכת המבוססת על תנועה טבעית. המערכת שנבנתה משלבת מספר עולמות הנדסיים: חישה אינרציאלית, תקשורת אלחוטית בתדר 2.4 GHz, מידול מכני בתלת-ממד ובקרה על מנועי זרם ישר. הדוח מפרט את התהליך ההנדסי, החל מהגדרת הפרוטוקולים ועד לפתרון בעיות אינטגרציה מורכבות.

3. רקע תאורטי

מערכת הבקרה מבוססת על שילוב של מספר טכנולוגיות מפתח:

א. מיקרו-בקר Arduino Uno

המוח המרכזי המבצע דגימה של אותות (קלט), עיבוד המידע ושליחת פקודות תנועה בזמן אמת. הבקר מתקשר עם רכיבי הקצה באמצעות שני פרוטוקולים:

- פרוטוקול I^2C : משמש לתקשורת עם חיישן ה-MPU6050 באמצעות שני קווים (SDA, SCL).
- פרוטוקול SPI: פרוטוקול מהיר המשמש להעברת נתונים למודול ה-NRF24L01.

ב. חיישן אינרציאלי MPU6050

חיישן המשלב מד-תאוצה וגירוסקופ. מד-התאוצה מודד תאוצה קווית בשלושה צירים. עקרון הפעולה מבוסס על מדידת היטל וקטור כוח הכובד (g) על צירי החיישן בעת הטיה. ערכי התאוצה הגולמיים מתורגמים לזוויות Roll ו-Pitch לצורך זיהוי כיוון הנסיעה.

ג. תקשורת אלחוטית NRF24L01

מודול הפועל בתדר 2.4 GHz. הוא מאפשר ניהול חבילות נתונים אוטומטי הכולל אימות הגעת המידע (ACK). השימוש בספריית NRFLite אפשר עבודה עם מבני נתונים (Structs) המבטיחים סנכרון מלא בין המשדר למקלט.

ד. בקר מנועים (H-Bridge) L298N

רכיב זה מאפשר שליטה דו-כיוונית בזרם המנוע. באמצעות אותות לוגיים מהארדואינו, הבקר קובע את כיוון סיבוב המנוע ומהירותו, תוך הפרדה בין מתח הבקרה (5V) למתח ההנעה מהסוללות.

4. מהלך הניסוי

א. רכיבי המערכת

- **בקרים:** 2 יחידות Arduino Uno.
- **תקשורת:** 2 מודולי NRF24L01.
- **חיישן:** MPU6050.
- **מכניקה:** שלד רכב בעיצוב עצמי בהדפסת תלת-ממד.
- **הנעה:** בקר L298N ושני מנועי DC.
- **אנרגיה:** 4 סוללות AA להנעה וסוללת 9V למשדר.

ב. מידול השלד ומערכת האנרגיה

שלד הרכב תוכנן בתוכנת מידול ממוחשב כדי לאפשר קיבוע מדויק של המנועים ובית הסוללות על מבנה קל משקל. מערכת האנרגיה הופרדה כך שהמנועים מוזנים מ-4 סוללות AA, בעוד הבקר והרדיו מקבלים מתח מיוצב למניעת ניתוקים כתוצאה מרעשים חשמליים.

ג. אלגוריתם המשדר

המשדר דוגם את החיישן, מבצע כיול, ומחליט על פקודה לפי ספי רגישות (Thresholds):

1. **דגימה:** פנייה ל-MPU6050 דרך I^2C .
2. **פענוח:**
 - הטייה בציר X מעל או מתחת ל-8000 מתורגמת לקדימה/אחורה.
 - הטייה בציר Y מעל או מתחת ל-8000 מתורגמת לימינה/שמאלה.
3. **שידור:** אריזת המידע ב-Struct ושליחה אלחוטית.

פסאודו-קוד משדר:

```
mpu.Execute()
if (rawX < -8000) command = 1 // forward
else if (rawX > 8000) command = 2 // backward
else if (rawY < -8000) command = 3 // left
else if (rawY > 8000) command = 4 // right
else command = 0 // stop
_radio.send(command)
```

ד. אלגוריתם המקלט

המקלט מאזין לערוץ הרדיו ומפעיל את המנועים רק בעת שינוי בפקודה:

1. **קליטה:** בדיקה אם יש מידע ב-`_radio.hasData()`.
2. **ייעול:** השוואת הפקודה ל-`lastCommand` למניעת הצפת המעבד.
3. **ביצוע:** שימוש ב-`switch-case` להפעלת המנועים.

פסאודו-קוד מקלט:

```
if (_radio.hasData()) {
    _radio.readData(&_radioData);
    if (_radioData.command != lastCommand) {
        moveAccordingly(_radioData.command); // 0-4
        lastCommand = _radioData.command;
    }
}
```

5. תוצאות וניתוח

המערכת עמדה ביעדים בהצלחה. השלד המודפס בתלת-ממד סיפק חוזק מכני ודיוק שאפשרו נסיעה ישרה ללא סטיות. נמצא כי סף הרגישות של ± 8000 הוא אופטימלי: הוא מסנן רעידות טבעיות של היד אך מגיב במהירות להטיה מכוונת. מבחינת אנרגיה, 4 סוללות AA סיפקו מתח מספק לנסיעה רציפה, והפרדת קווי המתח מנעה את "קפיאת" מודול הרדיו בזמן האצה. זיהינו כי ההשהיה של 100ms במשדר מאפשרת שליטה חלקה ללא איבוד חבילות נתונים.

6. קשיים

- **מכניקה:** בגרסאות הראשונות של השלד המודפס, החורים למנועים לא היו מדויקים, מה שגרם לחיכוך ולחוסר תנועה. הפתרון היה עדכון המודל והדפסה חוזרת.
- **תוכנה:** המקלט נטה לקפוא בגלל הצפת הודעות ב-Serial. פתרנו זאת על ידי מעבר לבדיקת if במקום לולאות חוסמות ועדכון רק בשינוי מצב.
- **חומרה:** חיישן ה-MPU6050 לא זוהה בתחילה בגלל בעיות חיווט. הטמענו קוד "סורק כתובות" ב-setup המזהה את החיישן פיזית לפני תחילת העבודה.
- **חשמל:** המנועים גרמו לנפילות מתח ששיבשו את הרדיו. פתרנו זאת באמצעות ייצוב המתח ושימוש בסוללות AA טעונות במלואן.

7. סיכום ומסקנות

הפרויקט שילב תכנון מכני, אלקטרוניקה ותוכנה לכדי תוצר עובד. למדנו כי מידול מוקדם של השלד חוסך בעיות הרכבה משמעותיות, וכי ניהול זרימת הנתונים (כמו השימוש ב-lastCommand) הוא קריטי לייצוב מערכות בזמן אמת. הפרויקט חזק את הידע שלנו בפרוטוקולי תקשורת (I^2C , SPI) ובהתמודדות עם רעשים חשמליים במערכות הכוללות מנועים.

8. כיווני המשך

- הוספת בקרת מהירות (PWM) לפי זווית ההטיה.
- הוספת חיישן מרחק למניעת התנגשות אוטונומית.

9. נספחים – קודי המערכת

9.1. קוד המשדר (TSM.ino)

```

/*
  This program detects the tilt state of the MPU6050 and sends a
  value
  to the receiver via NRF24L01. Includes visual indication via the
  built-in LED (pin 13) after calibration.

  Wiring for Arduino Uno:
  MPU6050: VCC->5V, GND->GND, SDA->A4, SCL->A5
  NRF24L01: VCC->3.3V only!, CE->7, CSN->8, SCK->13, MOSI->11, MISO
    ->12
*/

#include <Wire.h>
#include "TinyMPU6050.h"
#include <SPI.h>
#include "NRFLite.h"

MPU6050 mpu(Wire);

const static uint8_t RADIO_ID = 1;
const static uint8_t DESTINATION_RADIO_ID = 0;
const static uint8_t PIN_RADIO_CE = 7;
const static uint8_t PIN_RADIO_CSN = 8;
const int STATUS_LED = 13;

struct RadioPacket {
  uint8_t FromRadioId;
  uint32_t Data;
  uint32_t FailedTxCount;
};

NRFLite _radio;
RadioPacket _radioData;

void setup() {
  Serial.begin(9600);
  pinMode(STATUS_LED, OUTPUT);

```

```

digitalWrite(STATUS_LED, LOW);

Wire.begin();
delay(1000);
Serial.println("\n=== SYSTEM STARTUP ===");

// I2C bus scan
Serial.print("Scanning I2C bus... ");
Wire.beginTransmission(0x68);
byte error = Wire.endTransmission();
if (error == 0) {
    Serial.println("FOUND MPU6050 (0x68)!");
} else {
    Serial.println("NOT FOUND!");
    Serial.println("CRITICAL ERROR: Check SDA(A4) and SCL(A5) wires.");
}

Serial.println("Initializing Library...");
mpu.Initialize();
Serial.println("Calibrating... Keep sensor STILL.");
mpu.Calibrate();
Serial.println("Calibration Done!");

// Visual indicator: 5 fast blinks after calibration
for (int i = 0; i < 5; i++) {
    digitalWrite(STATUS_LED, HIGH); delay(100);
    digitalWrite(STATUS_LED, LOW); delay(100);
}

if (!_radio.init(RADIO_ID, PIN_RADIO_CE, PIN_RADIO_CSN)) {
    Serial.println("NRF24 Error: Check wiring of pins 7, 8, 11, 12, 13");
    while (1);
}

_radioData.FromRadioId = RADIO_ID;
Serial.println("Ready to send data.\n");
}

void loop() {

```



```

mpu.Execute();

long rawX = mpu.GetRawAccX();
long rawY = mpu.GetRawAccY();

if (rawX == -1 && rawY == -1) {
    Serial.println("ERROR: Sensor Disconnected!");
    digitalWrite(STATUS_LED, LOW);
    delay(500);
    return;
}

Serial.print("Current State: ");

if (rawX <= -8000) {
    _radioData.Data = 1;
    Serial.print("FRONT");
    Serial.print(" | X: "); Serial.println(rawX);
    digitalWrite(STATUS_LED, HIGH);
} else if (rawX >= 8000) {
    _radioData.Data = 2;
    Serial.print("BACK");
    Serial.print(" | X: "); Serial.println(rawX);
    digitalWrite(STATUS_LED, HIGH);
} else if (rawY <= -8000) {
    _radioData.Data = 3;
    Serial.print("LEFT");
    Serial.print(" | Y: "); Serial.println(rawY);
    digitalWrite(STATUS_LED, HIGH);
} else if (rawY >= 8000) {
    _radioData.Data = 4;
    Serial.print("RIGHT");
    Serial.print(" | Y: "); Serial.println(rawY);
    digitalWrite(STATUS_LED, HIGH);
} else {
    _radioData.Data = 0;
    Serial.print("STOP");
    Serial.print(" | X: "); Serial.print(rawX);
    Serial.print(" Y: "); Serial.println(rawY);
    digitalWrite(STATUS_LED, LOW);
}

```

```

    sendData();
}

void sendData() {
    if (!_radio.send(DESTINATION_RADIO_ID, &_radioData, sizeof(
        _radioData))) {
        Serial.println(" [!] No ACK from Receiver");
        _radioData.FailedTxCount++;
    } else {
        Serial.println(" [OK]");
    }
    delay(100);
}

```

9.2. קוד המקלט (RCV.ino)

```

/*
  This program receives messages from the transmitter and drives
  the car accordingly. Adapted for Arduino Uno.

  Motor wiring:
  Left motor:  forward -> 2, backward -> 3
  Right motor: forward -> 4, backward -> 5

  NRF24L01: VCC->3.3V, CE->7, CSN->8, SCK->13, MOSI->11, MISO->12
*/

#include <SPI.h>
#include "NRFLite.h"

const int leftMotorForward  = 2;
const int leftMotorBackward = 3;
const int rightMotorForward  = 4;
const int rightMotorBackward = 5;

const static uint8_t RADIO_ID = 0;
const static uint8_t PIN_RADIO_CE  = 7;
const static uint8_t PIN_RADIO_CSN = 8;

struct RadioPacket {

```

```

uint8_t FromRadioId;
uint32_t Data;
uint32_t FailedTxCount;
};

NRFLite _radio;
RadioPacket _radioData;

int directionOfMovement = 0;
int lastCommand = -1;

void setup() {
    Serial.begin(9600);

    pinMode(leftMotorForward,    OUTPUT);
    pinMode(leftMotorBackward,   OUTPUT);
    pinMode(rightMotorForward,   OUTPUT);
    pinMode(rightMotorBackward,  OUTPUT);

    if (!_radio.init(RADIO_ID, PIN_RADIO_CE, PIN_RADIO_CSN)) {
        Serial.println("Cannot communicate with radio");
        while (1);
    }

    Serial.println("Receiver Ready! Waiting for command...");
}

void loop() {
    // Using if instead of while to prevent blocking on overflow
    if (_radio.hasData()) {
        _radio.readData(&_radioData);
        directionOfMovement = _radioData.Data;

        // Print only on change – saves CPU time and prevents freezing
        if (directionOfMovement != lastCommand) {
            Serial.print("New Command Received: ");
            Serial.println(directionOfMovement);
            moveAccordingly();
            lastCommand = directionOfMovement;
        }
    }
}

```

```
    delay(5);
}

void moveAccordingly() {
    switch (directionOfMovement) {
        case 1: front(); break;
        case 2: back(); break;
        case 3: left(); break;
        case 4: right(); break;
        case 0: none(); break;
    }
}

void front() {
    digitalWrite(leftMotorForward, HIGH);
    digitalWrite(rightMotorForward, HIGH);
    digitalWrite(leftMotorBackward, LOW);
    digitalWrite(rightMotorBackward, LOW);
}

void back() {
    digitalWrite(leftMotorBackward, HIGH);
    digitalWrite(rightMotorBackward, HIGH);
    digitalWrite(leftMotorForward, LOW);
    digitalWrite(rightMotorForward, LOW);
}

void left() {
    digitalWrite(leftMotorForward, LOW);
    digitalWrite(rightMotorForward, HIGH);
    digitalWrite(leftMotorBackward, LOW);
    digitalWrite(rightMotorBackward, LOW);
}

void right() {
    digitalWrite(leftMotorForward, HIGH);
    digitalWrite(rightMotorForward, LOW);
    digitalWrite(leftMotorBackward, LOW);
    digitalWrite(rightMotorBackward, LOW);
}
```

```
void none() {  
    digitalWrite(leftMotorForward, LOW);  
    digitalWrite(rightMotorForward, LOW);  
    digitalWrite(leftMotorBackward, LOW);  
    digitalWrite(rightMotorBackward, LOW);  
}
```